AutConnect File Structure and User Flow Map

This document explains how the project folders are organized and how the main user flows map to files. The app uses Next.js App Router, so folders under app/ are primarily URL routes, not role folders. To find code for a user action, start with the route page.tsx, then follow imported components and server actions.

Folder Mental Model

- app/ contains screens and URL routes. A page.tsx file is the entry screen for that route, while nearby components support that screen.
- app/actions/ contains server actions that mutate data, such as login, account creation, note creation, and supervisor client management.
- app/api/ contains API route handlers, including Google OAuth and insights.
- app/components/ contains reusable UI pieces shared across routes.
- app/dashboard/ is the main dashboard hub. It mixes provider and supervisor dashboard UI because both roles enter through /dashboard.
- app/portal/notes/ is the shared note writing and note reading flow for providers and supervisors.
- app/portal/supervisor/ contains supervisor-only management screens, such as creating clients and editing care teams.
- src/lib/ contains shared backend utilities such as database access, session cookies, and scoring.
- prisma/ contains the database schema and migrations.
- public/ contains static assets. Future SVG assets should be organized separately under public/assets/ or as React components under app/components/icons/.

Create Account Flow

- Entry route: app/create-account/page.tsx
- Manual account action: app/actions/createProvider.ts
- Google button: app/components/auth/OAuthButtons.tsx
- Google start route: app/api/auth/google/start/route.ts
- Google callback route: app/api/auth/callback/google/route.ts and app/api/auth/google/callback/route.ts
- Session helper: src/lib/session.ts
- Database helper: src/lib/db.ts
- Purpose: The user either fills out the create account form or starts Google OAuth. Both paths create or update a Provider, set the autconnect_session cookie, and send the user toward dashboard or complete-profile.

Login Flow

- Entry route: app/login/page.tsx
- Manual login action: app/actions/loginProvider.ts
- Google login button: app/components/auth/OAuthButtons.tsx
- Google OAuth routes: app/api/auth/google/start/route.ts, app/api/auth/callback/google/route.ts, app/api/auth/google/callback/route.ts
- Session helper: src/lib/session.ts
- Purpose: Manual login verifies email/password and Google login verifies Google identity. Both end with the same session shape: providerId and role in autconnect_session.

Dashboard Flow

- Main route: app/dashboard/page.tsx
- Dashboard layout: app/dashboard/layout.tsx
- Provider carousel: app/dashboard/ClientCarousel.tsx
- Provider/client orbit: app/dashboard/ClientOrbit.tsx
- Orbit wrapper: app/dashboard/OrbitView.tsx
- Supervisor dashboard: app/dashboard/SupervisorDashboard.tsx
- Database/session helpers: src/lib/db.ts and src/lib/session.ts
- Purpose: /dashboard is the role-switching hub. It reads the session, loads assigned clients and notes, then renders either provider-style orbit UI or supervisor dashboard UI based on session.role.

Provider Note Writing Flow

- User clicks plus in: app/dashboard/ClientOrbit.tsx
- Add note route: app/portal/notes/add/page.tsx
- Overlay wrapper: app/portal/notes/add/NoteOverlay.tsx
- Interactive form: app/portal/notes/add/EntryForm.tsx
- Create note action: app/actions/createEntry.ts
- Validation schema: app/libs/schemas/entry.ts
- Item parser: app/libs/entryItems.ts
- Scoring helper: src/lib/sessionScoring.ts
- Database models: prisma/schema.prisma
- Purpose: The add note page prepares client, provider, goals, and existing entry data. EntryForm submits to createEntry, which writes Entry and AssessmentItem records and resets EntryRead rows so assigned providers see unread note signals.

Read Notes Flow

- Sphere click source: app/dashboard/ClientOrbit.tsx
- Dashboard overlay loader: app/dashboard/page.tsx
- Standalone note route: app/portal/notes/view/page.tsx
- Note viewer UI: app/portal/notes/view/NoteHistoryViewer.tsx
- Database model: EntryRead in prisma/schema.prisma
- Purpose: A note sphere opens recent notes for a selected client/provider. Opening notes creates or updates EntryRead rows, where readAt null means unread and a timestamp means read.

Supervisor Dashboard Flow

- Main route: app/dashboard/page.tsx
- Supervisor dashboard UI: app/dashboard/SupervisorDashboard.tsx
- Workspace shell: app/dashboard/SupervisorWorkspaceLayout.tsx
- Client grid: app/dashboard/GridClientView.tsx
- Add existing client view: app/dashboard/AddClientView.tsx
- Remove client view: app/dashboard/RemoveClientView.tsx
- Supervisor mutations: app/actions/manageSupervisorClients.ts
- Purpose: Supervisors enter through the same /dashboard route, but session.role selects SupervisorDashboard. The supervisor menu switches between grid, add existing client, remove client, and selected-client orbit modes.

Supervisor Client Creation Flow

- Create client route: app/portal/supervisor/create-client/page.tsx
- Provider selection form: app/portal/supervisor/create-client/CreateClientProvidersForm.tsx

- Create with providers action: app/actions/createClientWithProviders.ts
- Basic create client action: app/actions/createClient.ts
- Purpose: This creates a new Client and links providers through ProviderClient rows. It is separate from dashboard add-link behavior, which links the supervisor to an existing client.

Supervisor Care Team Flow

- Care team list route: app/portal/supervisor/edit-providers/page.tsx
- Care team list UI: app/portal/supervisor/edit-providers/EditProvidersClient.tsx
- Specific client route: app/portal/supervisor/edit-providers/[clientId]/page.tsx
- Specific client form: app/portal/supervisor/edit-providers/[clientId]/EditClientProvidersForm.tsx
- Assignment action: app/actions/manageSupervisorClients.ts
- Purpose: Supervisors choose a client and update assigned providers. The main mutation is updateClientAssignments, which edits Client and ProviderClient records.

Supervisor Insights Flow

- Insights trigger/UI: app/dashboard/SupervisorDashboard.tsx and app/dashboard/InsightsOverlay.tsx
- Client hook: app/dashboard/useInsights.ts
- API route: app/api/insights/route.ts
- Insight builder: app/api/insights/insightsEngine.ts
- Purpose: The supervisor opens insights for the active client. The UI calls /api/insights, and the server builds insight data from entries and related records.

Design and SVG Organization Recommendation

- Current static SVGs: public/file.svg, public/globe.svg, public/next.svg, public/vercel.svg, public/window.svg
- Recommended static assets folder: public/assets/
- Recommended static SVG folder: public/assets/icons/ and public/assets/illustrations/
- Recommended React SVG components folder: app/components/icons/
- Purpose: Keep route files focused on screens and user flow, while icons and visual assets live in predictable design folders. This will make future motion, React Spring, shadcn, and custom SVG work easier to find and maintain.

How To Find Code By User Flow

- Start with the URL the user is on.
- Open that folder under app/ and find page.tsx.
- Read imports from page.tsx to find supporting components.
- For data changes, follow form action imports into app/actions/.
- For session or database behavior, check src/lib/session.ts, src/lib/db.ts, and prisma/schema.prisma.
- For shared note behavior, check app/portal/notes/ first.
- For supervisor management behavior, check app/portal/supervisor/ and app/dashboard/SupervisorDashboard.tsx.